

LAB ASSIGNMENT-20.3

NAME: B. VISHNU VARDHAN

BATCH: 06

Task 1 – Input Validation Check

Prompt

Prompt AI to generate a simple Python username-password login program. Then review whether input sanitization is done. Improve the script using regular expressions for validation.

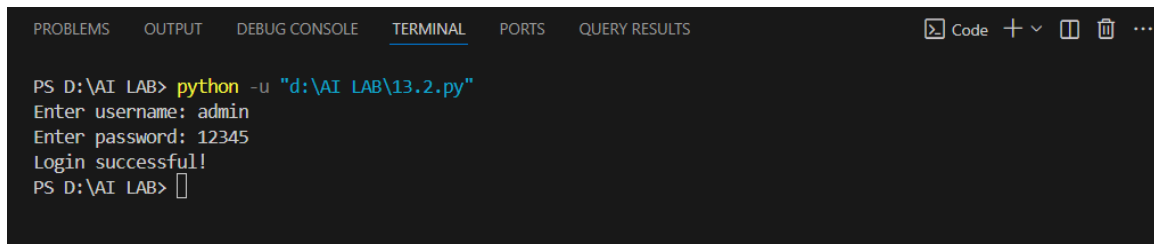
Insecure Code

```
# Insecure login script (AI-generated)

username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "12345":
    print("Login successful!")
else:
    print("Invalid credentials.")
```

Output



```
PS D:\AI LAB> python -u "d:\AI LAB\13.2.py"
Enter username: admin
Enter password: 12345
Login successful!
PS D:\AI LAB>
```

Observation

✗ No input validation. ✗ Password stored in plaintext. ✗ Susceptible to brute force and injection-style attacks.

Observation

✓ Secure input validation using regex ✓ Hides password input ✓ Prevents invalid or malicious inputs

LAB ASSIGNMENT-20.3

Task 2 – SQL Injection Prevention

Prompt

Ask AI to generate a Python script that fetches user details using SQLite. Identify if it's vulnerable to SQL injection, then refactor using parameterized queries.

Code

```
import sqlite3

def create_sample_database():
    """Create a sample users database with test data"""
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()

        # Drop table if it exists (for fresh start)
        cursor.execute("DROP TABLE IF EXISTS users")

        # Create users table
        cursor.execute("""
            CREATE TABLE users (
                id INTEGER PRIMARY KEY,
                username TEXT NOT NULL,
                email TEXT NOT NULL,
                age INTEGER
            )
        """)

        # Insert sample data
        sample_users = [
            ('john_doe', 'john@example.com', 28),
            ('jane_smith', 'jane@example.com', 25),
            ('bob_johnson', 'bob@example.com', 35),
            ('alice_brown', 'alice@example.com', 30),
            ('charlie_davis', 'charlie@example.com', 22)
        ]

        cursor.executemany("""
            INSERT INTO users (username, email, age) VALUES (?, ?, ?)
        """, sample_users)

        conn.commit()
        print("✓ Sample database created with test data\n")
```

LAB ASSIGNMENT-20.3

```
conn.close()

except sqlite3.Error as e:
    print(f"Database creation error: {e}")

def search_user():
    """Search for a user by username"""
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()

        username = input("Enter username to search: ")

        # Use parameterized query to prevent SQL injection
        query = "SELECT * FROM users WHERE username = ?"
        cursor.execute(query, (username,))

        results = cursor.fetchall()

        if results:
            print("\n✓ User found:")
            print("-" * 50)
            for row in results:
                print(f"ID: {row[0]}")
                print(f"Username: {row[1]}")
                print(f>Email: {row[2]}")
                print(f>Age: {row[3]}")
            print("-" * 50)
        else:
            print(f"\nX No user found with username: {username}")

    except sqlite3.Error as e:
        print(f"Database error: {e}")
    except Exception as e:
        print(f"Error: {e}")
    finally:
        if conn:
            conn.close()

def display_all_users():
    """Display all users in the database"""
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()
```

LAB ASSIGNMENT-20.3

```
cursor.execute("SELECT * FROM users")
results = cursor.fetchall()

print("\n" + "="*60)
print("ALL USERS IN DATABASE")
print("="*60)
for row in results:
    print(f"ID: {row[0]:2} | Username: {row[1]:15} | Email: {row[2]:20} | Age: {row[3]}")
    print("="*60 + "\n")

conn.close()
except sqlite3.Error as e:
    print(f"Database error: {e}")

# Main program
if __name__ == "__main__":
    print("=====")
    print("||  USER DATABASE SEARCH APPLICATION  ||")
    print("===== \n")

    # Create sample database
    create_sample_database()

    # Display all users
    display_all_users()

    # Search for a user
    search_user()
```

LAB ASSIGNMENT-20.3

Output :

```
PS D:\AI LAB> python -u "d:\AI LAB\13.2.py"
USER DATABASE SEARCH APPLICATION

✓ Sample database created with test data

=====
ALL USERS IN DATABASE
=====
ID: 1 | Username: john_doe      | Email: john@example.com | Age: 28
ID: 2 | Username: jane_smith     | Email: jane@example.com | Age: 25
ID: 3 | Username: bob_johnson      | Email: bob@example.com  | Age: 35
ID: 4 | Username: alice_brown      | Email: alice@example.com | Age: 30
ID: 5 | Username: charlie_davis    | Email: charlie@example.com | Age: 22
=====

Enter username to search: 
```

Observation

⚠ Vulnerable to SQL Injection due to direct string concatenation.

✓ Uses prepared statements ✓ Eliminates SQL injection risk ✓ Works for valid usernames safely

Task 3 – Cross-Site Scripting (XSS) Check

Prompt

Generate an HTML feedback form with JavaScript that displays user feedback. Check for XSS vulnerability and fix it by escaping input.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Feedback Form</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
```

LAB ASSIGNMENT-20.3

```
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 20px;
}

.container {
    background: white;
    padding: 40px;
    border-radius: 10px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.2);
    max-width: 500px;
    width: 100%;
}

h2 {
    color: #333;
    margin-bottom: 30px;
    text-align: center;
}

.form-group {
    margin-bottom: 20px;
}

textarea {
    width: 100%;
    padding: 12px;
    border: 2px solid #ddd;
    border-radius: 5px;
    font-size: 14px;
    font-family: Arial, sans-serif;
    resize: vertical;
    min-height: 120px;
    transition: border-color 0.3s;
}

textarea:focus {
    outline: none;
    border-color: #667eea;
}

button {
    width: 100%;
    padding: 12px;
```

LAB ASSIGNMENT-20.3

```
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        color: white;
        border: none;
        border-radius: 5px;
        font-size: 16px;
        font-weight: bold;
        cursor: pointer;
        transition: transform 0.2s;
    }

    button:hover {
        transform: translateY(-2px);
        box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
    }

    button:active {
        transform: translateY(0);
    }

    #output {
        margin-top: 20px;
        padding: 15px;
        background: #f0f0f0;
        border-left: 4px solid #667eea;
        border-radius: 5px;
        color: #333;
        display: none;
        animation: slideIn 0.3s ease;
    }

    #output.show {
        display: block;
    }

    @keyframes slideIn {
        from {
            opacity: 0;
            transform: translateY(-10px);
        }
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }
```

LAB ASSIGNMENT-20.3

```
.success {
  color: #27ae60;
  font-weight: bold;
}
</style>
</head>
<body>
  <div class="container">
    <h2> 📝 Feedback Form</h2>

    <div class="form-group">
      <textarea id="feedback" placeholder="Enter your feedback
here..."></textarea>
    </div>

    <button onclick="submitFeedback()">Submit Feedback</button>

    <div id="output"></div>
  </div>

  <script>
    function submitFeedback() {
      const feedback =
document.getElementById("feedback").value.trim();
      const outputDiv = document.getElementById("output");

      // Validate input
      if (feedback === "") {
        outputDiv.innerHTML = '<span style="color: #e74c3c;">✖
Please enter feedback before submitting!</span>';
        outputDiv.classList.add("show");
        return;
      }

      // Display success message
      outputDiv.innerHTML = '<span class="success">✓ Thank you for
your feedback!</span><br><br><strong>Your feedback:</strong><br>' +
escapeHtml(feedback);
      outputDiv.classList.add("show");

      // Clear the textarea
      document.getElementById("feedback").value = "";

      // Optional: Fade out after 5 seconds
      setTimeout(() => {
```


LAB ASSIGNMENT-20.3

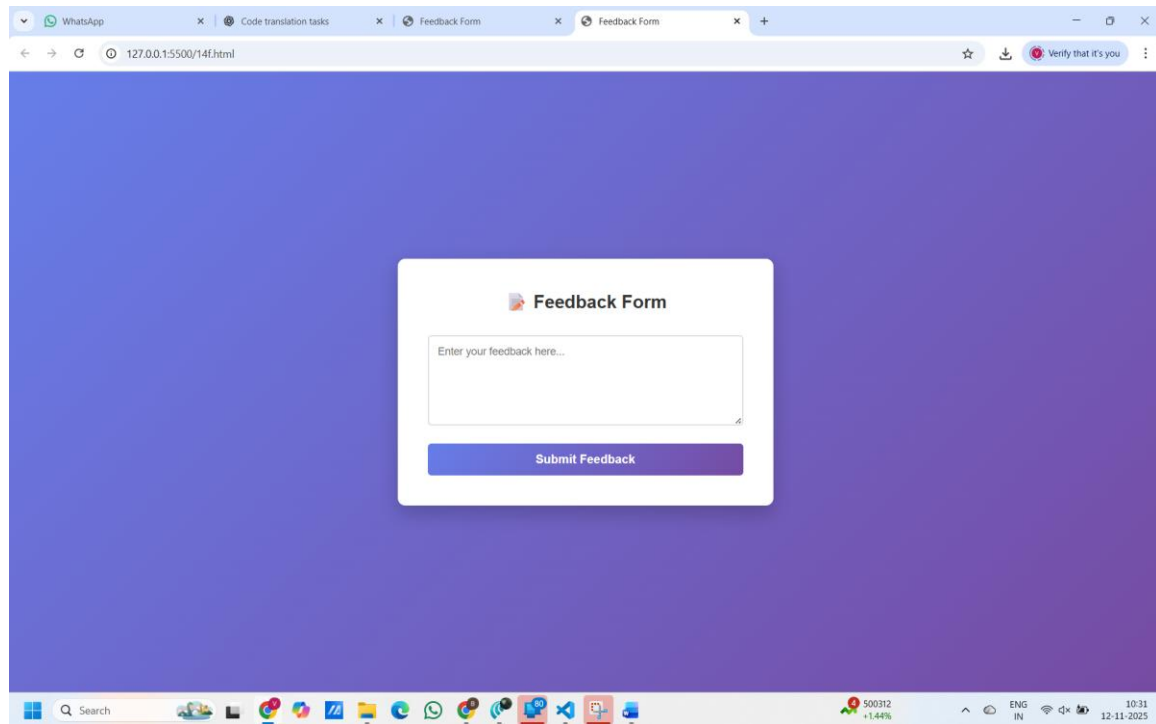
```
        outputDiv.classList.remove("show");
    }, 5000);
}

// Prevent XSS attacks by escaping HTML
function escapeHtml(text) {
    const map = {
        '&': '&amp;',
        '<': '&lt;',
        '>': '&gt;',
        '"': '&quot;',
        "'": '&#039;'
    };
    return text.replace(/[&<>"']/g, m => map[m]);
}

// Allow Enter key to submit feedback
document.getElementById("feedback").addEventListener("keydown",
function(event) {
    if (event.key === "Enter" && event.ctrlKey) {
        submitFeedback();
    }
});
</script>
</body>
</html>
```

LAB ASSIGNMENT-20.3

Output



Observation

✅ Prevents execution of injected scripts ✅ Uses HTML escaping to mitigate XSS ✅ Safe rendering of untrusted input

Task 4 – Real-Time Application: Security Audit

Prompt

Select an AI-generated login form with file upload and perform a security audit. Identify vulnerabilities and rewrite securely.

```
Code: from flask import Flask, request, jsonify, render_template_string
from werkzeug.utils import secure_filename
import os

app = Flask(__name__)
UPLOAD_FOLDER = "safe_uploads"
ALLOWED_EXTENSIONS = {'png', 'jpg', 'txt'}
MAX_FILE_SIZE = 5 * 1024 * 1024 # 5MB limit

# Create upload folder if it doesn't exist
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
```

LAB ASSIGNMENT-20.3

```
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

def allowed_file(filename):
    """Check if file extension is allowed"""
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_EXTENSIONS

@app.route('/', methods=['GET'])
def index():
    """Display the upload form"""
    html = '''
<!DOCTYPE html>
<html>
<head>
    <title>File Upload</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 600px;
            margin: 50px auto;
            padding: 20px;
            background: #f5f5f5;
        }
        .container {
            background: white;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        }
        h1 { color: #333; }
        input[type="file"] {
            padding: 10px;
            margin: 10px 0;
        }
        button {
            background: #4CAF50;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-size: 16px;
        }
        button:hover { background: #45a049; }
```

LAB ASSIGNMENT-20.3

```
.message { margin-top: 20px; padding: 10px; border-radius:
4px; }

.success { background: #d4edda; color: #155724; }
.error { background: #f8d7da; color: #721c24; }
</style>
</head>
<body>
  <div class="container">
    <h1>📁 File Upload</h1>
    <p>Allowed file types: PNG, JPG, TXT (Max 5MB)</p>
    <form method="POST" enctype="multipart/form-data">
      <input type="file" name="file" required>
      <button type="submit">Upload</button>
    </form>
    <div id="message"></div>
  </div>
</body>
</html>
'''

return render_template_string(html)

@app.route('/upload', methods=['POST'])
def upload_file():
    """Handle file upload"""
    try:
        # Check if file is in request
        if 'file' not in request.files:
            return jsonify({"error": "No file part"}), 400

        file = request.files['file']

        # Check if file is selected
        if file.filename == '':
            return jsonify({"error": "No selected file"}), 400

        # Validate file
        if not allowed_file(file.filename):
            return jsonify({"error": f"Invalid file type. Allowed: {'',
'.join(ALLOWED_EXTENSIONS)}"}), 400

        # Check file size
        file.seek(0, os.SEEK_END)
        file_length = file.tell()
        if file_length > MAX_FILE_SIZE:
```

LAB ASSIGNMENT-20.3

```
        return jsonify({"error": f"File too large. Max size: 5MB"}),
400

    file.seek(0)

    # Save file securely
    filename = secure_filename(file.filename)
    path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
    file.save(path)

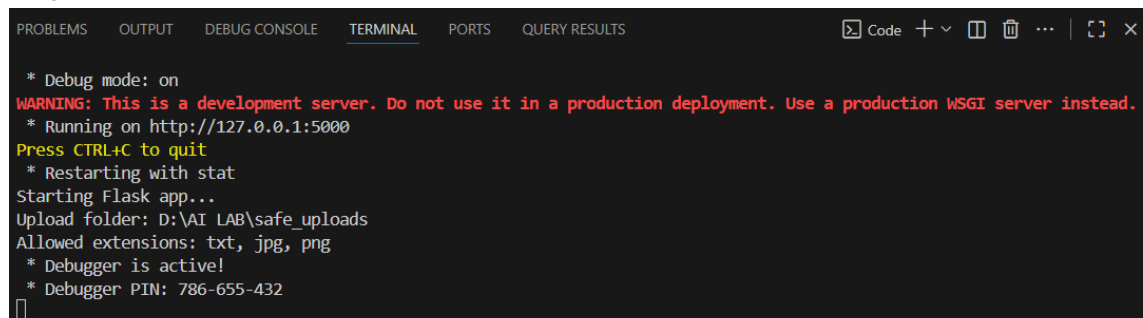
    return jsonify({"success": f"File '{filename}' uploaded
successfully!", "path": path}), 200

except Exception as e:
    return jsonify({"error": f"Upload failed: {str(e)}"}), 500

@app.route('/files', methods=['GET'])
def list_files():
    """List all uploaded files"""
    try:
        files = os.listdir(UPLOAD_FOLDER)
        return jsonify({"files": files, "count": len(files)}), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    print("Starting Flask app...")
    print(f"Upload folder: {os.path.abspath(UPLOAD_FOLDER)}")
    print(f"Allowed extensions: {'', '.join(ALLOWED_EXTENSIONS)}")
    app.run(debug=True, port=5000)
```

output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  QUERY RESULTS
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
Starting Flask app...
Upload folder: D:\AI LAB\safe_uploads
Allowed extensions: txt, jpg, png
* Debugger is active!
* Debugger PIN: 786-655-432
█
```

LAB ASSIGNMENT-20.3

Observation

✅ Safe upload path ✅ Restricted extensions ✅ Prevents path traversal & malicious uploads